

How Well Can LLMs Interpret Network Configurations?

A Benchmark and Tool-Augmented Multi-Agent Approach

Hyeonjeong Lee, Yujin Park, and Cheoneum Park, *Member, IEEE*

Abstract—LLM은 네트워크 설정 파일을 얼마나 정확하게 해석하고 분석할 수 있는가? 기존 벤치마크는 표준 문서의 지식 회상이나 설정 생성에 치중하여 이 질문에 답하기 어렵다. 본 연구는 벤치마크, 평가 지표, 도구 통합 시스템을 하나의 프레임워크로 제안한다. NetConfigQA 2.0은 설정 파일과 Batfish 시뮬레이션을 결합한 이중 경로 파이프라인을 통해 5단계 인지 난이도의 QA 쌍을 자동 생성하며, 설정 파일과 정책 파일만 교체하면 토폴로지 규모에 관계없이 데이터셋이 확장된다. 답변 타입별 정답 구조를 반영하는 Type-Aware Accuracy(TA-Acc)를 도입하여 네트워크 데이터의 평가 문제를 다룬다. 4개 토폴로지(10~40 노드, 총 9,462 QA)와 6개 LLM을 평가한 결과, 사실 추출(L1~L3)에서는 모델 간 차이가 뚜렷하나 경로 추론과 장애 분석(L4/L5)에서는 모든 모델이 시뮬레이션 장벽에 직면하였다. 이 한계를 다루기 위해 도구 통합 Multi-Agent 시스템 NetAlly를 제시하고, Single LLM → Pure MAS → NetAlly의 3-way 비교를 통해 성능 향상의 주된 원인이 에이전트 구조가 아닌 네트워크 분석 도구의 통합에 있음을 보인다.

Index Terms—네트워크 구성, 대규모 언어 모델, Multi-Agent 시스템, 벤치마크, 정형 검증, Batfish, Type-Aware Accuracy.

I. INTRODUCTION

규모 언어 모델(LLM)은 자연어 이해, 코드 생성, 다단계 추론 등에서 높은 성능을 보이며 다양한 실무 태스크로 빠르게 확장되고 있다 [?]. 그러나 설정 파일, 실행 환경, 외부 분석 도구가 함께 얹히는 운영 태스크에서는 단일 LLM만으로 해결하기 어려운 한계가 남아 있다. 특히 외부 도구와의 연동, 동적 환경 인식, 긴 컨텍스트에 걸친 정밀 추론은 단순한 텍스트 생성보다 더 복잡한 조정 과정을 요구한다. 이러한 배경에서 여러 에이전트가 역할을 분담해 태스크를 처리하는 Multi-Agent System(MAS) 접근이 확산되고 있으며 [?], LangGraph [?] 등의 프레임워크가 에이전트 간 상태 관리와 도구 호출을 체계화하고 있다.

네트워크 관리 분야 역시 이러한 흐름의 영향을 직접적으로 받고 있다 [?], [?]. 최근에는 도메인 특화 사전학습 [?], LLM 생성 코드 기반 자동화 [?], 네트워크 태스크 적응 프레임워크 [?] 등 다양한 접근이 제안되었고, NetConfEval [?], NIKA [?], NetPress [?], 그리고 IETF의 NetConfBench 초안 [?]은 평가 방법론을 구체화하고 있다. 이는 네트워크 분야에서도 더 이상 LLM의 가능성 자체를 묻는 단계에 머물지 않고, 어떤 태스크를 어떤 방식으로 평가하고 보완할 것인가가 중요한 문제로 이동하고 있음을 보여준다.

그러나 대부분의 기존 벤치마크는 지식 검색(Knowledge Retrieval)에 집중한다. TeleQnA [?]는 3GPP 표준 문서에서 사실을 회상하는지, NetBench [?]는 전문가 쿼레이션 지식을 활용하는지를 평가한다. 이는 실제 설정 파일에서 네트워크가 어떻게 동작하는지를 추론하는 동작 추론(Behavioral Inference)과는 별개의 문제이다. .cfg 파일에 “PE1의 OSPF cost가 10”이라고 명시된 것을 읽는 것(L1)과, 그 설정에 따라 PE1에서 PE2까지의 실제 패킷 경로를 추론하는 것(L4)은 질적으로 구분되는 인지 능력을 요구한다.

이 질문에 체계적으로 답하려면 세 가지 조건이 필요하다. 첫째, 사실 추출부터 경로 추론, 장애 영향 분석까지 인지 난이도를 구분하는 벤치마크가 있어야 한다. 기존 벤치마크는 데이터 플레인 시뮬레이션 능력을 측정하지 않는다. 둘째, 네트워크 데이터에 적합한 평가 지표가 필요하다. BERTScore는 10.0.1.1과 10.0.1.10에 0.9 이상의 유사도를 부여하고, Exact Match는 {r1, r2}와 {r2, r1}을 오답으로 처리하는 등 구조화된 네트워크 답변에 부적합하다. 셋째, 토폴로지의 크기와 복잡도에 따라 LLM의 해석 능력이 달라지므로, 매번 수작업으로 데이터셋을 구성하는 대신 설정 파일만으로 QA를 자동 생성하는 파이프라인이 요구된다.

본 연구는 하나의 연구 질문을 중심으로 전개된다: **LLM은 네트워크 설정 파일을 얼마나 정확하게 해석하고 분석할 수 있는가?** 이에 답하기 위해 먼저 벤치마크를 구축하고, 평가 과정에서 드러난 한계를 도구 통합 시스템으로 다루며, 그 결과를 실증적으로 분석한다.

본 연구의 기여는 다음과 같다.

- **C1.** 벤치마크, 자동 생성 파이프라인, 평가 지표. 실제 설정 파일로부터 5단계 인지 난이도의 QA 쌍을 자동 생성하는 벤치마크 NetConfigQA 2.0을 구축한다. 이중 경로 (Dual-Path) 파이프라인은 설정 파일과 정책 파일만 교체하면 토폴로지 규모에 관계없이 데이터셋을 자동 확장하며, 네트워크 데이터의 정답 구조를 반영하는 평가 지표 TA-Acc를 함께 제안한다.
- **C2.** 도구 통합 Multi-Agent 시스템과 기여 분리 실험. 벤치마크 평가에서 드러난 시뮬레이션 장벽을 다루기 위해 PNETLab, Cisco NSO, Batfish 세 평면을 통합하는 Multi-Agent 시스템 NetAlly를 제안하고, Single LLM → Pure MAS → NetAlly의 3-way 비교로 구조 효과와 도구 효과를 분리한다.
- **C3.** 실증 분석. 6개 LLM × 4개 토폴로지(10~40 노드, 총 9,462 QA)에 대한 평가를 통해 시뮬레이션 장벽을

H. Lee, Y. Park, and C. Park are with the Department of Computer Science, Chungnam National University, Daejeon, South Korea (e-mail: {hjlee, yjpark, cheoneum}@cnu.ac.kr).

Manuscript received [TODO]; revised [TODO].

관찰하고, 성능 변화의 원인을 체계적으로 분석한다.

Section II에서는 관련 연구를 정리한다. Section III에서는 벤치마크 NetConfigQA 2.0과 TA-Acc를 기술하고, Section IV에서는 NetAlly 시스템을 설명한다. Section V에서는 실험 결과를 제시하며, Section VI에서는 그 의미와 한계를 논의한다. 마지막으로 Section VII에서 결론을 맺는다.

II. RELATED WORK

A. 네트워크 LLM 벤치마크

Table I은 기존 네트워크 LLM 벤치마크를 평가 대상별로 정리한 것이다.

지식 회상(Knowledge Recall). LLM이 표준 문서의 내용을 얼마나 정확히 재현하는지를 측정한다. TeleQnA [?]는 3GPP 문서에서 추출한 다지선다 질문으로 통신 도메인 지식을 평가하며, TeleQuAD [?]는 동일 문서에서 구간 추출(Span Extraction) 기반의 읽기 이해를 측정한다. NetBench [?]는 전문가가 큐레이션한 서술형 질문에 대한 LLM의 답변을 전문가 패널이 평가한다. 이들 벤치마크는 LLM의 도메인 지식 보유 여부를 확인하는데 유용하나, 특정 네트워크 토폴로지에서의 동작을 추론하는 능력은 평가하지 못한다.

설정 생성(Configuration Generation). LLM이 올바른 설정 파일을 작성할 수 있는지를 평가한다. NetConfEval [?]은 주어진 요구 사항으로부터 라우터 설정을 생성하도록 하고, Batfish로 구문 및 의미 정확성을 검증한다. NetComplete [?]은 SMT 기반으로 부분 설정을 자동 완성하는 합성 접근을 제안하였다. NETLLMBENCH [?]는 YANG 스키마 기반의 설정 생성과 에뮬레이션 기반 검증을 결합한다. 이 흐름은 설정 작성 능력을 평가하는 반면, 본 연구는 기존 설정을 읽고 추론하는 능력에 초점을 맞춘다.

동적 추론(Dynamic Reasoning). 네트워크의 런타임 동작을 추론하는 능력을 평가하며, 본 연구가 속한다. NetPress [?]는 런타임에 네트워크 상태를 생성하고 LLM에게 프로토콜 동작을 예측하도록 하여, 정확도와 안전성을 동시에 측정한다. 본 연구의 전신인 NetConfigQA [?]는 단일 토폴로지에서 설정 해석 QA를 제시하였으나, 인지 난이도 체계와 시뮬레이션 기반 질문(L4/L5)이 부재하였다. NetConfigQA 2.0은 이를 확장하여 실제 설정 파일로부터의 5단계 인지 난이도 평가, 4개 토폴로지에 걸친 자동 QA 생성 파이프라인, 그리고 네트워크 데이터에 특화된 평가 지표(TA-Acc)를 제공한다.

B. LLM 기반 네트워크 관리 에이전트

LLM을 네트워크 운영에 적용하는 연구는 단발성 질의 변환에서 다단계 자율 추론까지 폭넓게 분포한다.

사용자 질문을 도구 쿼리로 번역하는 단발성 접근이 가장 단순한 형태이다. AskBatfish [?]는 자연어 질문 하나를 Batfish API 호출 하나로 변환하며, 다단계 분석이나 도구 간 연계는 지원하지 않는다.

인테트 기반 접근은 운영자의 의도를 설정 변경으로 변환한다. INTA [?]는 RAG 기반 에이전트로 인테트를 네트워크 설정으로 변환하여 98.15%의 구문 정확도를 달성하였다. Confucius [?]는 Multi-Agent 구조로 인테트 기반 네트워크 관리의 자동화를 제안하였다. 이들은 설정 작성 태스크를 다루며, 기존 설정으로부터의 추론은 범위 밖이다.

Multi-Agent 구조를 통한 성능 개선도 보고되고 있다. KubeLLM [?]은 Knowledge Agent와 Tools Agent를 분리하여 단일 에이전트 대비 Kubernetes 관리 정확도를 개선하였다. NIKA [?]는 5개 시나리오, 640개 이상의 인시던트를 갖춘 평가 아레나로, MCP 인터페이스를 통해 30개 이상의 도구를 노출하며 GPT-4급 모델의 장애 탐지 능력과 근본 원인 식별의 한계를 보고하였다. 그러나 이들 연구에서 보고된 성능 향상이 에이전트 구조 자체에서 기인하는지, 연결된 도구에서 기인하는지는 분리되지 않았다.

NetAlly는 단발성 변환이 아닌 다단계 자율 추론을 수행한다는 점에서 기존 연구와 구별된다. 에이전트가 토폴로지 분석에서 fork_snapshot, 차등 도달성 검사, 영향 범위 도출에 이르는 연쇄적 도구 호출을 자율적으로 구성하며, Batfish(정형 검증), NSO(구성 관리), PNETLab(시뮬레이션)이라는 세 개의 독립된 관리 평면을 하나의 파이프라인에 통합한다. 또한 3-way 비교 실험을 통해 구조와 도구의 기여를 분리 관찰한다.

C. 연구 위치

기존 연구는 벤치마크와 시스템을 독립적으로 제공한다. TeleQnA, NetConfEval 등은 벤치마크를 제공하되 한계를 극복할 시스템은 제시하지 않으며, NIKA, INTA 등은 시스템을 제공하되 통제된 벤치마크 위에서의 체계적 평가가 부족하다.

본 연구는 이 간극을 메우기 위해 벤치마크와 시스템을 하나의 프레임워크 안에서 연결한다. 먼저 NetConfigQA 2.0과 TA-Acc로 문제를 정의하고, Single LLM 평가를 통해 한계를 확인한다. 이어서 Pure MAS 실험으로 구조 효과를 분리하고, 도구 통합 시스템 NetAlly를 통해 그 한계를 어떻게 우회할 수 있는지 평가한다. 이 설계를 통해 “MAS를 도입하면 성능이 향상된다”는 기존 보고를 반복하는 데 그치지 않고, 그 향상의 원인이 구조인지 도구인지까지 분석할 수 있다.

III. NETCONFIGQA 2.0: 벤치마크 데이터셋

A. 토폴로지 설계

기본 토폴로지(Lab-A)는 10대의 Cisco IOS 라우터로 구성된 Service Provider MPLS VPN 네트워크이다. P(Provider) 라우터 4대, PE(Provider Edge) 라우터 2대, Leaf 스위치 4대가 OSPF, MP-BGP, LDP, VRF 구성으로 연결되어 있다. 파이프라인의 확장성을 검증하기 위해 National Converged Network(NCN) 시리즈를 설계하였다. 동일한 파이프라인과 정책 파일(policies.json)을 사용하면서 토폴로지만 교체하여, 설정 복잡도의 증가에 따른 데이터셋 자동 확장을 입증한다.

모든 설정 파일은 PNETLab 에뮬레이션 환경에서 배포 및 검증되었다. Lab-B~D의 설정은 YAML 토폴로지 정의와 Jinja2 템플릿 기반 Config Generator로 자동 생성하였으며, OSPF/BGP/LDP 프로토콜 수렴을 확인하였다. Lab-D에는 LLM의 정합성 검증 능력을 평가하기 위해 3가지의 도적 설정 오류를 삽입하였다: (1) PE8의 VRF_ISP_BROKEN에 route-target을 완전 누락, (2) PE7→PE8 iBGP 피어링 비대칭 누락, (3) FW1→PE7 OSPF cost=10, PE7→FW1 cost=20의 비대칭 설정. 이 오류들은 각각 vrf_without_rt, ibgp_missing_pairs, asymmetric_ospf_cost 메트릭에 의해 탐지되어야 한다.

TABLE I
네트워크 LLM 벤치마크 비교

벤치마크	태스크 유형	데이터 소스	GT 생성	평가 지표
TeleQnA [?]	지식 회상	3GPP 문서	인간+LLM	Accuracy
TeleQuAD [?]	지식 추출	3GPP 규격	구간 추출	F1 / EM
NetBench [?]	지식 서술	전문가	전문가 작성	전문가 평가
NetConfEval [?]	설정 생성	문서	참조 설정	Accuracy
NETLLM BENCH [?]	생성+검증	스키마	에뮬레이션	구문+의미
NetPress [?]	동적 추론	런타임 생성	에뮬레이터	정확도+안전
Ours	설정 해석+동작 추론	실제 .cfg	규칙 기반 + Batfish 시뮬레이션	TA-Acc

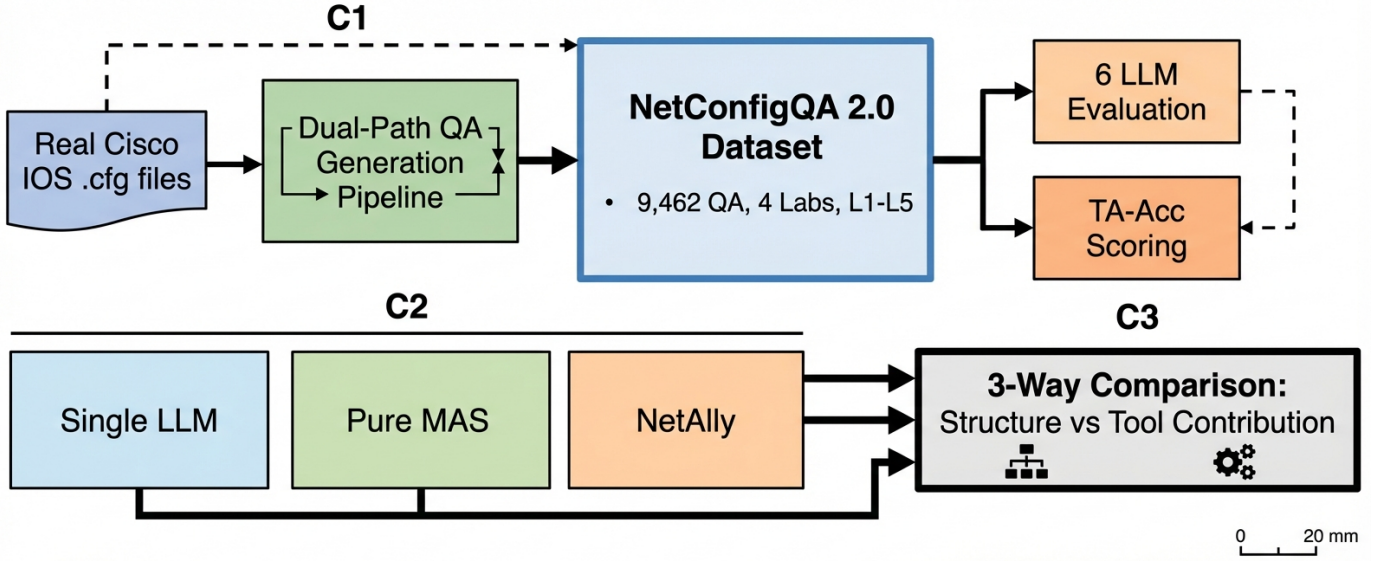


Fig. 1. 연구 프레임워크 개요. 설정 파일로부터 QA를 자동 생성하는 벤치마크와 평가 지표로 LLM의 해석 능력을 평가하고(C1), 이 장벽을 우회하기 위한 도구 통합 시스템을 제시하며(C2), 3-way 비교를 통해 구조 효과와 도구 효과를 분리 분석한다(C3).

TABLE II
토폴로지 요약

Lab	노드	컨셉	핵심 프로토콜	QA 수
Lab-A	10	SP MPLS VPN	OSPF, BGP, LDP, VRF	1,264
Lab-B	20	NCN 기본 SP	+ NTP, SNMP, AAA, Banner	2,154
Lab-C	30	NCN 보안 + L2VPN	+ L2VPN, ACL, eBGP, HSRP	2,673
Lab-D	40	NCN 멀티 AS 복합	+ QoS, NetFlow, 3-AS, Waypoint	3,371
합계	—	—	—	9,462

B. 이중 경로(Dual-Path) QA 생성 파이프라인

NetConfigQA 2.0은 두 가지 경로로 QA 쌍을 생성한다.

경로 A (L1~L3, 규칙 기반): 설정 파일을 Batfish [?]가 파싱하여 정적 팩트(Static Facts) JSON을 생성한다. policies.json에 정의된 127개 메트릭은 네트워크 운영자가 토폴로지에 대해 일상적으로 확인하는 항목(인터페이스 상태, 라우팅 이웃, ACL 규칙 등)을 Cisco IOS 운영 매뉴얼과 Batfish가 지원하는 분석 범위를 기준으로 체계화한 것이다. 각 메트릭에 대해 질문 템플릿, 스코프 확장 규칙,

답변 타입이 지정되어 있다. 스코프 확장(Scope Expansion) 메커니즘은 12가지 스코프 타입(GLOBAL, DEVICE, DEVICE_PAIR, AS, OSPF_AREA, VRF, FLOW 등)에 걸쳐 질문 인스턴스를 동적으로 생성한다. 이 과정에서 하나의 메트릭이 토폴로지 내 모든 해당 엔티티에 대해 자동으로 인스턴스화되므로, 노드 수 증가에 비례하여 QA가 확장된다.

템플릿 기반 생성에서 질문 형태의 단조로움은 스코프 확장 메커니즘(장비명, VRF명, 인터페이스명의 동적 치환)과, L4/L5의 조합적 노드 쌍 샘플링을 통해 완화된다. 후자

는 키워드 오버피팅이 아닌 토폴로지 구조 이해를 정답 도출의 전제 조건으로 만든다.

Table ??는 127개 메트릭의 17개 카테고리별 구성과 대표 질문 템플릿을 정리한 것이다. L1 카테고리(System/Security/Interface/Routing/Services Inventory)는 단일 장비의 팩트를 추출하며, L2~L3 카테고리(Security_Policy, OSPF/L2VPN/BGP/VRF_Consistency, Comparison)는 복수 장비 간의 집계 또는 정합성을 검증한다. L4~L5 카테고리(Reachability/What-If_Analysis)는 Batfish 시뮬레이션을 통한 동작 추론을 요구한다.

경로 B (L4~L5, 절차적): Batfish 시뮬레이션 엔진을 활용한다. L4에서는 노드 쌍을 샘플링하여 traceroute와 reachability 쿼리를 실행하고, L5에서는 fork_snapshot으로 지정된 링크/노드를 비활성화한 가상 네트워크 복사본을 생성한 뒤 differentialReachability 쿼리로 장애 영향을 분석한다. 시뮬레이션 결과가 자연어 질문-답변 쌍으로 자동 변환된다.

C. 5단계 인지 난이도 체계

Table III은 네트워크 운영자의 인지 부하(Cognitive Load)를 기준으로 설계한 5단계 난이도 체계이다. L1~L3는 설정 파일의 텍스트에서 직접 답을 도출할 수 있는 정적 분석 영역이며, L4~L5는 설정에 필요한 정보가 모두 포함되어 있으나 그래프 알고리즘(OSPF SPF 등)의 실행이 요구되는 동작 추론 영역이다.

Lab-A의 레벨별 분포와 4개 Lab의 비교는 Table IV에 제시한다. L1은 장비 단위 메트릭(hostname, 인터페이스 IP 등)으로 구성되어 메트릭 수가 고정되므로, Lab-B 이후에서는 새 프로토콜이 추가되어도 L1 질문 수가 일정하다. 반면 L4는 노드 쌍 조합이 $O(N^2)$ 로 증가하므로 146(Lab-A) → 1,657(Lab-D)로 급증하며, 파이프라인이 토폴로지 복잡도에 자동으로 적응함을 보여준다. L2는 GLOBAL 스코프 메트릭("전체 장비 중 SSH 활성화 비율")으로 구성되어 노드 수에 관계없이 일정하며, 프로토콜 종류 변화에 따라 소폭 감소한다.

D. Type-Aware Accuracy (TA-Acc)

전통적 NLP 지표는 네트워크 구성 평가에 부적합하다. BERTScore는 의미적으로 다른 IP 주소(10.0.1.1과 10.0.1.10)에 0.9 이상의 유사도를 부여하여 모델 간 차이를 변별하지 못한다. Exact Match는 집합의 순서 차이($\{r1, r2\}$ vs $\{r2, r1\}$)를 오답으로 처리한다. Token F1은 경로의 순서를 무시하여 역방향 경로($PE1 \rightarrow P1 \rightarrow R1$ 과 $R1 \rightarrow P1 \rightarrow PE1$)를 정답으로 인정한다.

TA-Acc는 각 질문의 답변 타입 t_i 에 따라 적합한 비교 함수 f_{t_i} 를 선택하여 개별 점수를 산출하고, 전체 데이터셋에 대해 산술 평균을 취한다.

정의. 데이터셋 $D = \{(q_i, g_i, t_i)\}_{i=1}^N$ 이 주어지고, 모델의 예측을 p_i 라 할 때, TA-Acc는 다음과 같이 정의된다:

$$TA-Acc(D) = \frac{1}{N} \sum_{i=1}^N f_{t_i}(\hat{p}_i, \hat{g}_i) \quad (1)$$

여기서 $\hat{p}_i$ 와 $\hat{g}_i$ 는 각각 예측과 정답에 정규화(소문자 변환, CIDR 표기 제거, 널 값 통합 등)를 적용한 결과이다. 비교 함수 f_t 는 Table V에 정의된 6가지 타입별로 다르게 적용된다.

set과 map 타입은 F1 Score를 사용하여 0과 1 사이의 연속 점수를 반환한다. 구체적으로, 예측 집합 P 와 정답 집합 G 에 대해:

$$f_{set}(P, G) = \frac{2 \cdot |P \cap G|}{|P| + |G|} \quad (2)$$

map 타입은 (키, 정규화된 값) 쌍을 집합 원소로 취급하여 동일한 F1을 적용한다. 나머지 4개 타입(text, path, number, boolean)은 이진 점수(0 또는 1)를 반환하되, 각 타입에 특화된 정규화를 수행한다. 특히 path 타입은 노드 순서를 보존하는 Ordered Exact Match를 적용하여, $R1 \rightarrow P1 \rightarrow PE1$ 과 $PE1 \rightarrow P1 \rightarrow R1$ 을 구분한다.

의도적으로 Token F1 fallback을 배제하였다. 네트워크 데이터에서 Token F1은 부분 일치에 과도한 점수를 부여하여(예: 경로의 일부 노드만 맞춰도 높은 점수), 모델 간 변별력을 저하시키기 때문이다.

집계 방식. 전체 QA에 대한 산술 평균(Micro Average)을 기본 지표로 사용하되, 레벨별 분포 불균형의 영향을 통제하기 위해 Level-Macro Average를 병행 보고한다. Level-Macro Average는 각 레벨의 TA-Acc를 먼저 산출한 뒤 5개 레벨의 산술 평균을 취한 것으로, L4 질문이 지배적인 대규모 토폴로지에서 Micro Average와의 괴리를 확인하는 데 유용하다.

E. Ground Truth 검증

Ground Truth는 Batfish [?]의 formal network analysis로 정의된다. Batfish는 설정 파일로부터 결정적(deterministic) 계산을 수행하는 네트워크 시뮬레이터로, Header Space Analysis [?], VeriFlow [?], Minesweeper [?] 등과 함께 네트워크 정형 검증 분야의 핵심 도구이다. NetConfEval [?]에서도 oracle로 사용된 바 있으며, Config2Spec [?]과 DNA [?]는 Batfish 위에서 설정 명세 추출 및 차등 분석을 수행하며, Champion [?]은 라우터 설정 쌍의 동작 차이를 자동 디버깅한다. Brown et al. [?]은 Batfish가 연구 프로토타입에서 산업용 도구로 진화한 과정을 정리하며, 설정 분석 도구의 신뢰성을 뒷받침한다. 파이프라인 정확성 검증을 위해 3가지 독립적 방법을 사용한다.

Method 1 (독립 구문 분석기): Batfish를 사용하지 않는 Python+Regex 기반 파서(약 2,100줄)가 설정 파일에서 L1~L3 정답을 독립적으로 재도출한다. 이 파서의 핵심 제약은 pybatfish를 일체 import하지 않는 것이며, 이 독립성이 순환 논증 방지의 근거가 된다. 동일한 .cfg 파일을 입력으로 사용하되, Batfish 엔진과 완전히 독립된 코드 경로로 정답을 재도출함으로써 생성 오라클 편향을 통제한다. 4개 Lab에서 총 5,719건을 전수 검증하였다.

“전체 일치율”이 Lab-B~D에서 92%대인 것은 GT 오류가 아니라 독립 파서의 검증 범위 한계에 기인한다. 시뮬레이션 의존 메트릭(L4/L5 traceroute 결과 등)은 Python+Regex 파서로 재도출이 원리적으로 불가능하여 불일치로 집계되지만, 이는 Method 3(PNETLab CLI)에서 별도로 검증한다. 불일치 원인을 분류하면 시뮬레이션 의존(57%), Batfish 파싱 한계(37%), 독립 파서 한계(6%)이며, 정적 팩트의 실질 불일치는 4개 Lab 합산 1건(< 0.02%)에 불과하다. 즉, “정적 검증 범위”(98~99.9%)가 L1~L3 GT의 실제 정확도를 나타내며, “전체 일치율”의 하락은 검증 방법의 한계이지 GT 품질의 저하가 아니다.

Method 2 (계층화 수동 검증): 각 Lab에서 레벨별로 층화 무작위 추출한 67~73개 표본에 대해, 원본 .cfg 파일을

TABLE III
메트릭 카테고리별 구성 및 대표 질문 템플릿 (L6 DIAGNOSTIC_TROUBLESHOOTING 1개 메트릭은 본 논문 범위 외로 제외)

#	카테고리	레벨	수	대표 질문 템플릿
1	System_Inventory	L1	11	"{host} 장비의 호스트네임은 무엇입니까?"
2	Configuration_Check	L1,L3	30	"{host} 장비의 Password Encryption 설정값은?"
3	Security_Inventory	L1	4	"{host} 장비의 SSH 버전은 무엇입니까?"
4	Interface_Inventory	L1	4	"{host} 장비에 설정된 네트워크 인터페이스는 총 몇 개입니까?"
5	Routing_Inventory	L1	7	"{host} 장비의 BGP Local-AS 번호는?"
6	Services_Inventory	L1	7	"{host} 장비에 설정된 VRF 이름 목록을 알려주세요."
7	Hardware_Inventory	L1	4	"네트워크 전체에서 Serial 인터페이스를 보유한 장비 목록은?"
8	Security_Policy	L2	7	"SSH 접속이 가능한 장비 목록을 알려주세요."
9	OSPF_Consistency	L2	3	"OSPF Area {area}에 속한 장비 목록은?"
10	L2VPN_Consistency	L2,L3	5	"구성된 L2VPN pseudowire 회선(장비쌍) 목록은?"
11	BGP_Consistency	L3	5	"AS {asn}의 iBGP Full-Mesh 상태와 누락된 피어링은?"
12	VRF_Consistency	L3	4	"route-target이 없는 VRF(장비/VRF) 목록은?"
13	Comparison_Analysis	L3	11	"{host1}과 {host2}의 BGP 피어 수를 비교하세요."
14	Routing_Consistency	L4	1	"{node1}과 {node2} 사이 OSPF 네이버 미형성 원인을 분석하세요."
15	Reachability_Analysis	L4	9	"{src_ip}에서 {dst_ip}까지의 네트워크 경로를 나열해주세요."
16	What-If_Analysis	L5	14	"{link} 링크 다운 시, {src}→{dst} 트래픽에 어떤 영향이 발생합니까?"
합계 (본 논문 범위)		L1~L5	126	

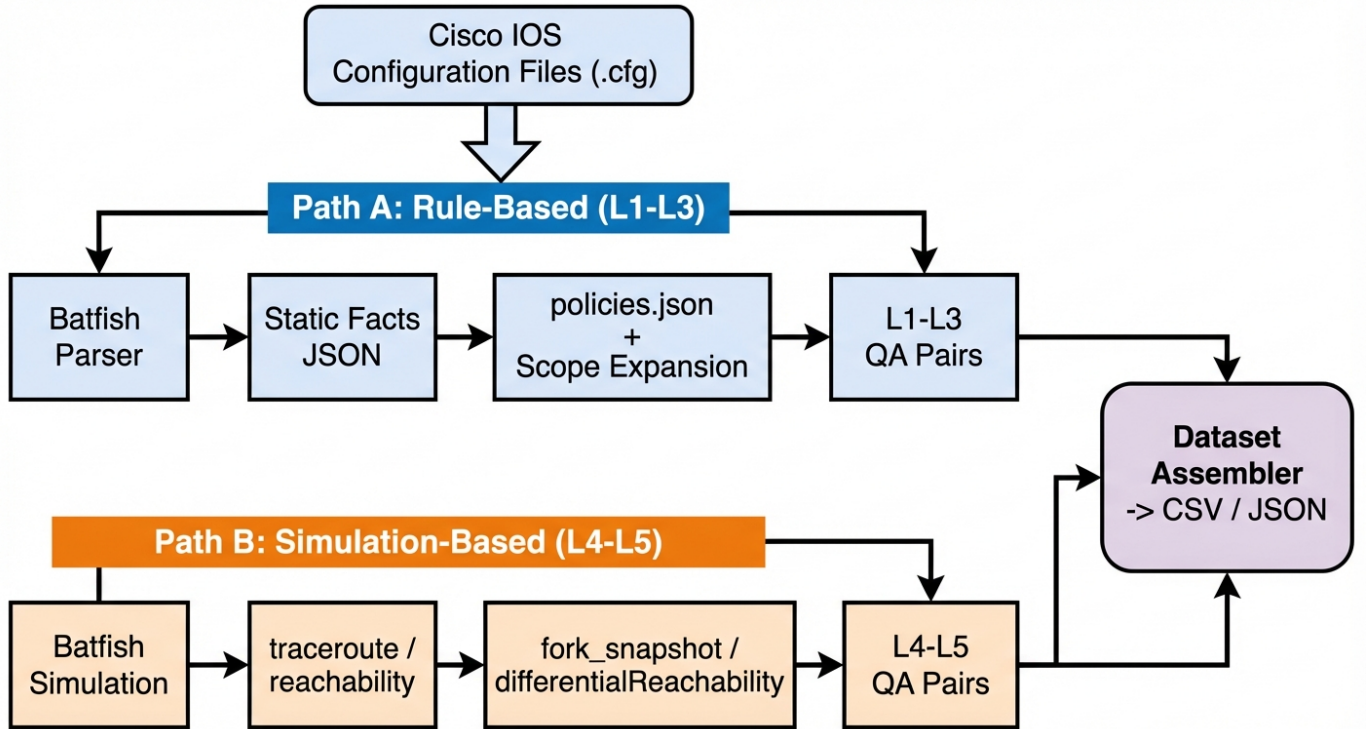


Fig. 2. NetConfigQA 2.0 이중 경로 QA 생성 파이프라인. 경로 A는 Batfish 파싱 → 정적 팩트 → 규칙 기반 생성(L1~L3), 경로 B는 Batfish 시뮬레이션 → traceroute/fork_snapshot → 절차적 생성(L4~L5).

직접 열어 질문-정답 쌍의 정확성을 교차 검증하였다. 검증 대상은 L1~L3 정적 팩트로 한정하며, 검증자는 해당 설정 구문을 직접 확인하여 Batfish 파싱 결과와의 일치 여부를 판정하였다. 4개 Lab에서 불일치 사례는 발견되지 않았다.

Method 3 (PNETLab 실장비 CLI): L4/L5 시뮬레이

션 기반 정답의 신뢰성을 검증하기 위해, Lab당 44~47개 표본에 대해 PNETLab 에뮬레이션 환경의 Cisco IOS CLI에서 `show ip route`, `traceroute`, `show ip ospf neighbor` 등의 명령어를 실행하여 Batfish 시뮬레이션 결과와 대조하였다. 실장비 CLI 출력과 Batfish 정답 간 불일치는 관찰되

TABLE IV
인지 난이도 체계

레벨	인지 활동	질문 본질	정답 생성 방식	참조 장비 수
L1	사실 추출	설정에 무엇이 적혀 있는가?	Config 파싱	$O(1)$
L2	통계 집계	네트워크 전체 통계는?	전체 장비 순회	$O(N)$
L3	정합성 비교	설정 간 논리적 모순은?	장비 간 교차 검증	$O(N^2)$
L4	경로 시뮬레이션	패킷이 실제로 도착하는가?	Batfish traceroute	프로토콜 시뮬레이션
L5	장애 영향 추론	장애 시 어떻게 되는가?	fork_snapshot + diff	복수 시뮬레이션

TABLE V
Lab별 레벨 분포

레벨	Lab-A	Lab-B	Lab-C	Lab-D
L1	660 (52.2%)	1,230	1,230	1,230
L2	104 (8.2%)	101	80	69
L3	252 (19.9%)	255	255	253
L4	146 (11.6%)	441	954	1,657
L5	102 (8.1%)	127	154	162
합계	1,264	2,154	2,673	3,371

TABLE VI
TA-Acc의 답변 타입별 비교 함수

답변 타입	비교 함수 f_t	적용 예시
set	Set F1 (순서 무관)	BGP 이웃 목록, 인터페이스 집합
text	정규화 후 EM	호스트네임, IP 주소
path	Ordered EM	traceroute 경로
number	숫자 추출 후 EM	홉 수, AS 번호, 카운트
boolean	의미 정규화 비교	SSH 활성화 여부
map	Key-Value F1	장비별 통계 비교

TABLE VII
GROUND TRUTH 검증 결과. “전체 일치율”은 시뮬레이션 의존 메트릭을 포함한 수치이며, “정적 검증 범위”는 이를 제외하고 독립 파서로 검증 가능한 범위만의 일치율이다.

Lab	검증 건수	전체 일치율	정적 검증 범위
Lab-A (10 노드)	1,016	99.9%	99.9%
Lab-B (20 노드)	1,586	92.3%	99.1%
Lab-C (30 노드)	1,565	92.8%	98.2%
Lab-D (40 노드)	1,552	92.5%	98.4%

지 않았으며, 이는 Batfish의 시뮬레이션이 해당 토폴로지 에서 실제 네트워크 동작을 정확히 재현함을 확인한다.

GT 독립성에 관한 사전 논의. L4/L5의 Ground Truth 와 Section IV의 NetAlly가 모두 Batfish에 의존하므로, NetAlly의 L4/L5 평가는 부분적으로 “올바른 Batfish API

를 올바른 파라미터로 호출했는가”를 측정하는 순환(circularity)이 존재한다. 이를 통제하기 위해 (1) Method 3에서 PNETLab 실장비 CLI로 Batfish 출력의 독립 검증을 수행 하였고, (2) 순환에서 자유로운 L1~L3에서도 동일한 성능 격차 패턴이 관찰되는지를 확인하며, (3) Section V에서 도구 호출 성공/실패별 TA-Acc를 분리 보고하여 “API 호출 성공”과 “추론 품질”을 구분한다. 이 제약의 상세 논의는 Section VI-D에서 다룬다.

IV. NETALLY: 네트워크 운영 MULTI-AGENT 시스템

A. 설계 동기: 왜 도메인 학습이 아닌 3-Plane 통합인가

L4/L5의 성능 격차를 해소하는 접근으로 도메인 특화 학습(fine-tuning)과 기존 분석 도구 통합이 있다. 우리는 후자를 채택하였다. 네트워크 설정 파일은 벤더와 OS 버전에 따라 구문이 상이하여 학습 데이터 확보가 어렵고, L4/L5 태스크는 그래프 알고리즘의 실행을 요구하므로 학습량을 늘려도 정확도 상한이 존재한다. 반면 Batfish와 같이 이미 검증된 분석 도구를 활용하면 정확도와 신뢰성을 동시에 확보할 수 있다.

B. 3-Plane 통합 아키텍처

NetAlly는 세 개의 독립된 네트워크 관리 평면을 하나의 Chat 인터페이스로 통합한다: (1) **PNETLab Plane** — 가상 토폴로지 관리, 장비 부팅, 콘솔 접근, (2) **NSO Plane** — RESTCONF 기반 구성 조회·동기화·변경, (3) **Batfish Plane** — 정형 검증, traceroute 시뮬레이션, What-If 분석. 에이전트는 Orchestrator-Executor 구조를 LangGraph [?] 기반 상태 기계(State Machine)로 구현하며, 질의의 성격에 따라 적절한 평면의 도구를 선택한다.

Orchestrator (계획 에이전트): 경량 LLM이 스킴 카탈 로그(각 스킴의 이름, 설명, 필요 도구)가 YAML 프론트매 터로 정의된 SKILL.md 파일 집합)와 사용자 질의를 입

력받아, JSON 형식으로 1~3개의 관련 스킬을 선택하고 선택 근거를 출력한다. 예를 들어, “P1-P2 링크 장애 시 PE1→Leaf3 경로는?”이라는 질의에 대해 Orchestrator는 `["core", "network_verify"]`를 선택하고, “What-If 분석이 필요하므로 Batfish fork_snapshot을 사용해야 한다”는 추론을 생성한다. 선택된 스킬의 상세 가이드(SKILL.md 본문)가 Executor의 시스템 프롬프트에 주입되어, 도구 호출의 맥락과 제약 조건을 전달한다.

Executor (실행 에이전트): Executor LLM은 Orchestrator가 주입한 스킬 프롬프트와 16개 이상의 MCP 도구 바인딩을 기반으로 ReAct(Reason-Act-Observe) 루프를 수행한다. 각 반복에서 대화 이력을 분석하여 다음 행동을 결정하고, 도구를 호출한 뒤 출력을 관찰하여 추가 호출 여부를 판단한다. 최종 답변이 도출되거나 최대 10단계에 도달하면 루프가 종료된다.

C. 다층 오류 복구 메커니즘

NetAlly는 다층 오류 복구(Multi-layer Error Recovery)를 통해 단순 도구 래퍼(Wrapper)와 차별화된다. 도구 호출 실패를 Python 예외로 전파하지 않고, 구조화된 오류 디서너리(`{"error": ..., "tool": ...}`)로 변환하여 LLM의 대화 이력에 삽입한다. 이를 통해 LLM은 다음 반복에서 (a) 다른 도구로 전환, (b) 동일 도구를 수정된 파라미터로 재호출, (c) 부분적 결과로 최선의 답변 생성 중 하나를 자율적으로 선택한다. 이 외에도 `async` 호출 실패 시 `sync` 스레드로의 자동 폴백, `NETALLY_MCP_ALLOW_MUTATIONS` 플래그를 통한 읽기 전용 모드 강제, 10회 반복 후 강제 종료를 통한 무한 루프 방지, 미포착 예외의 SSE error 이벤트 변환 등의 안전 장치를 포함한다. 예를 들어, Batfish가 특정 설정 파일을 파싱하지 못하는 경우에도 에이전트는 NSO 쿼리로 대안을 시도하며, 완전한 실패보다 부분적 답변을 우선한다.

D. 핵심 도구와 자율 파이프라인

Executor가 호출하는 핵심 도구는 세 가지이다: (1) `network_query`는 Cisco NSO RESTCONF를 통해 실시간 구성을 조회하며 L1~L3에 주로 활용된다. (2) `network_verify`는 Batfish를 통해 정형 검증과 What-If 분석을 수행하며 L4~L5에 활용된다. (3) `lab_manage`는 PNETLab 토폴로지 관리 및 자동 온보딩을 담당한다. NetAlly는 두 가지 자동화 파이프라인을 통해 수동 조작을 최소화한다.

자율 온보딩. Section V의 실험에서 4개 Lab의 설정 파일 수집과 Batfish 스냅샷 초기화에 사용된다. PNETLab의 가상 장비를 사람의 CLI 조작 없이 운영 가능 상태로 전환한다. 파이프라인은 (1) LabFS에서 토폴로지와 콘솔 포트를 파싱하고, (2) 각 장비에 Telnet으로 접속하여 SSH를 활성화하며, (3) NSO에 RESTCONF로 등록 후 설정을 동기화하고, (4) 수집된 설정으로 Batfish 스냅샷을 초기화한다. IOS 버전별 명령어 호환성, 관리 IP 충돌, 동기화 실패 시 자동 재시도 등의 예외 처리가 포함된다.

Fork-Snapshot 기반 What-If 분석. L5 장애 영향 추론을 지원하기 위해, Batfish의 `fork_snapshot`으로 지정된 변경(링크/노드 비활성화)을 적용한 가상 네트워크 복사본을 생성한다. 원본 스냅샷은 보존되므로 비교과적이다. 생성된 복사본에 `differentialReachability` 쿼리를 실행하여, 장애 전후의 도달성 차이를 식별하고 영향 범위를 도출

한다. 이 과정은 에이전트가 자율적으로 수행하며, 운영자는 “P1-P2 링크가 다운되면 어떤 영향이 있는가?”와 같은 자연어 질의만 제공하면 된다.

V. EXPERIMENTS

A. 실험 설정

평가 데이터셋은 NetConfigQA 2.0의 4개 Lab으로 구성되며, 전체 QA 수는 9,462개이다. 모델은 4개 벤더에서 6개 LLM을 선정하였다.

모든 평가는 zero-shot 조건에서 수행하며, temperature는 0으로 고정하였다. `num_ctx`는 49,152로 통일하였는데, 이는 가장 큰 Lab-D(40노드)의 전체 설정 파일이 약 31K 토큰을 차지하며, 여기에 시스템 프롬프트와 출력 여유를 더한 값이다. 모델별 기본 컨텍스트 길이(128K~262K)가 상이하므로, 공정한 비교를 위해 동일 컨텍스트 예산을 적용하였다. `MAX_OUTPUT_TOKENS`는 70,000으로 설정하였으며, 이는 일부 reasoning 모델이 내부 추론에 최대 19K 토큰을 사용하는 것이 관찰되었기 때문이다. 로컬 모델 서빙은 RTX 3090 (24GB) 환경의 Ollama를 사용하였다.

평가 지표는 TA-Acc와 Format Stability(Parse Success Rate)이다. Format Stability는 TA-Acc만으로는 구분할 수 없는 “모델이 답을 모르는 것”과 “답을 알지만 포맷을 맞추지 못하는 것”의 차이를 식별하기 위한 보조 지표이다. LLM의 원본 출력을 답변 타입별 파서에 통과시켜, 파싱 가능 여부를 사후 집계한다. 예를 들어 `set` 타입은 쉼표/줄바꿈 구분 리스트로 파싱 가능한지, `path` 타입은 노드 시퀀스(→ 구분자)로 파싱 가능한지를 기준으로 판정한다. 모델 간 성능 차이의 유의성은 Bootstrap 95% 신뢰구간(10,000회 리샘플링)으로 검정한다. Single LLM 평가가 완료되면 Overall TA-Acc가 가장 높은 오픈소스 모델을 Pure MAS와 NetAlly의 공통 backbone으로 사용한다([TODO]: 모델명).

B. Single LLM 기준선과 확장성

LLM의 설정 파일 해석 능력을 평가하기 위해 6개 모델을 4개 Lab에 각각 적용하여 총 $6 \times 4 = 24$ 회 평가하였다.

파이프라인 확장성은 QA/노드 비율로 함께 보고한다. 본 데이터셋에서는 Lab-A 126, Lab-B 108, Lab-C 89, Lab-D 84로, 토폴로지 규모가 증가해도 QA 생성량이 동일한 수준에서 유지됨을 확인할 수 있다.

Table IX은 Lab-A에서의 레벨별 TA-Acc를 정리하기 위한 표이다. 이 표와 Fig. 4는 모델이 어느 난이도 구간에서 성능 저하를 보이는지, 그리고 그 저하가 L3→L4 구간에 집중되는지를 점검하는 데 사용한다. 본 연구는 설정 파일에 경로 계산에 필요한 정보가 존재함에도 그래프 기반 계산이 요구되는 시점에서 나타나는 성능 절벽을 **시뮬레이션 장벽 (Simulation Barrier)**으로 지칭한다. 또한 결과 보고에서는 Micro와 Macro Average를 병행하여 레벨 불균형의 영향을 함께 제시한다. 예를 들어 Lab-A에서는 L4 비율이 11.6%로 낮아 두 지표의 차이가 제한적일 수 있지만, L4 비율이 49%인 Lab-D에서는 Micro Average가 L4에 더 크게 영향을 받을 수 있으므로 Macro Average를 함께 해석하는 것이 필요하다.

C. 구조 효과와 도구 효과: 3-Way 비교

도구 통합을 통한 시뮬레이션 장벽의 우회 가능성을 확인하기 위해, Section V-B에서 Overall TA-Acc가 가장 높은 오픈소스 모델을 선정하여 세 가지 설정을 비교한다.

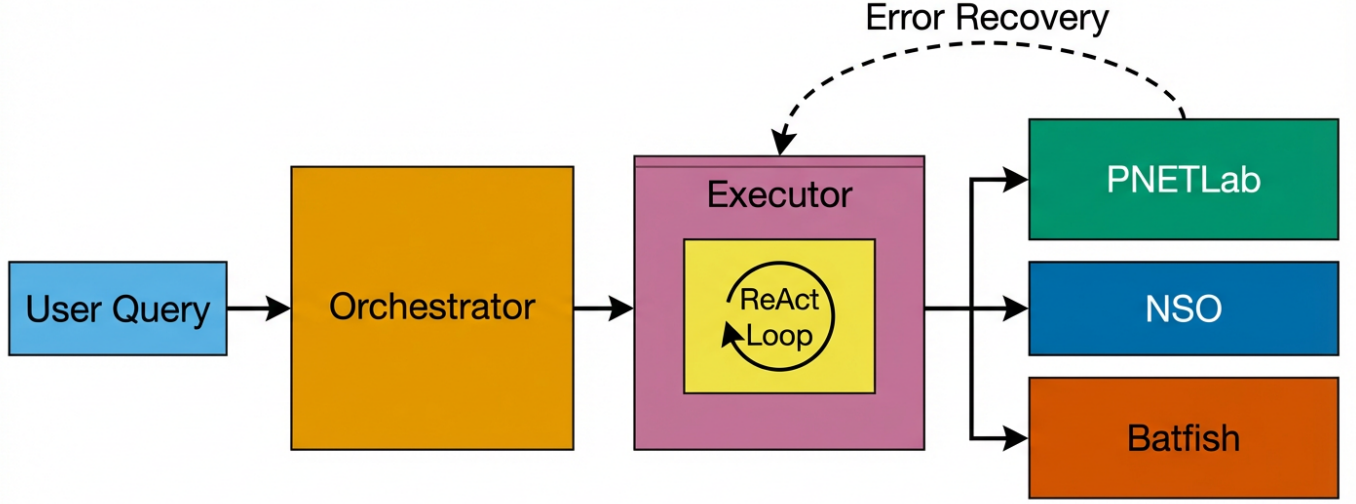


Fig. 3. NetAlly 3-Plane 통합 아키텍처. Orchestrator가 질의를 분석하여 스킬을 선택하고, Executor가 ReAct 루프를 통해 PNETLab/NSO/Batfish 평면의 도구를 자율적으로 호출한다.

TABLE VIII
평가 모델 구성

모델	벤더	파라미터	양자화	Context
GPT-4o-mini	OpenAI	– (API)	–	128K
GPT-OSS-20B	OpenAI	20B	MXFP4	131K
Qwen3-Coder	Alibaba	30B MoE / 3B active	Q4_K_M	256K
Gemma-3-27B	Google	27B	Q4_K_M	128K
GLM-4.7-Flash	Zhipu	30B MoE / 3B active	Q4_K_M	198K
Qwen3.5-27B	Alibaba	27B	Q4_K_M	262K

TABLE IX
SINGLE LLM 기준선: OVERALL TA-ACC (6개 모델 × 4개 Lab). 각 셀은 MICRO / MACRO 순이다.

모델	Lab-A (10)	Lab-B (20)	Lab-C (30)	Lab-D (40)
GPT-4o-mini	[TODO]	[TODO]	[TODO]	[TODO]
GPT-OSS-20B	[TODO]	[TODO]	[TODO]	[TODO]
Qwen3-Coder	[TODO]	[TODO]	[TODO]	[TODO]
Gemma-3-27B	[TODO]	[TODO]	[TODO]	[TODO]
GLM-4.7-Flash	[TODO]	[TODO]	[TODO]	[TODO]
Qwen3.5-27B	[TODO]	[TODO]	[TODO]	[TODO]

- **Single LLM:** 도구 없음, 설정 파일 전문을 프롬프트에 포함 (Section V-B 결과 재사용)
- **Pure MAS:** Orchestrator + Executor 구조, 모든 도구 비활성화. L4/L5 질문에 대해 Executor는 도구 호출 없이 프롬프트에 포함된 설정 파일만으로 경로 추론을 시도한다.
- **NetAlly:** Orchestrator + Executor + Batfish/NSO/

PNETLab 도구 활성화

세 설정의 공정한 비교를 위해 다음 변인을 통제한다: (1) 동일 LLM backbone (Section V-B에서 선정된 모델), (2) 동일 시스템 프롬프트 기본 구조 (Pure MAS와 NetAlly는 스킬 가이드 주입 여부만 상이), (3) 동일 최대 토큰 예산 및 최대 추론 단계 ($\text{max_steps} = 10$), (4) 동일 설정 파일 입력. Pure MAS와 Single LLM의 차이는 질문 분해 (Orchestrator의 스

TABLE X
LAB-A 레벨별 TA-ACC. MICRO는 전체 QA 평균, MACRO는 레벨별 평균의 산술 평균이다.

모델	L1	L2	L3	L4	L5	Micro	Macro
GPT-4o-mini	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]
GPT-OSS-20B	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]
Qwen3-Coder	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]
Gemma-3-27B	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]
GLM-4.7-Flash	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]
Qwen3.5-27B	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]

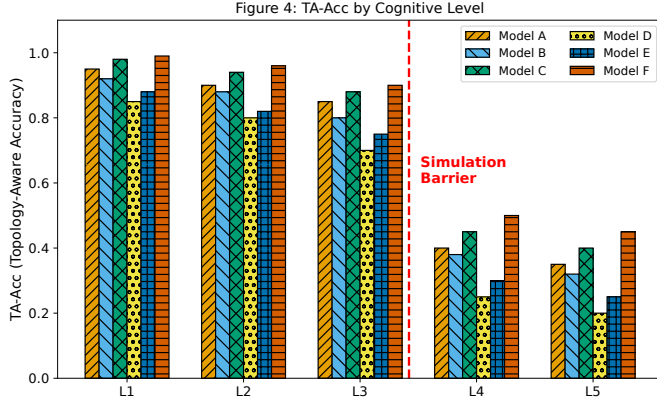


Fig. 4. Lab-A에서의 인지 난이도별 TA-Acc. 이 그림은 6개 모델의 레벨별 성능 분포를 비교하기 위한 것으로, 점선은 본 연구가 시뮬레이션 장벽으로 해석하는 L3→L4 경계를 나타낸다.

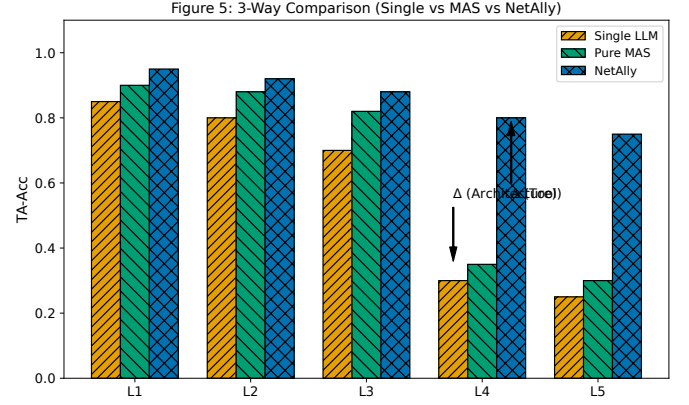


Fig. 6. Single LLM, Pure MAS, NetAlly의 3-way 비교. Δ (구조)는 Single→Pure MAS, Δ (도구)는 Pure MAS→NetAlly의 차이를 나타내며, 구조 효과와 도구 효과를 분리해 해석하기 위한 도식이다.

킬 선택)와 다단계 추론(Executor의 ReAct 루프)에 한정되며, 두 설정 모두 외부 도구에 접근할 수 없다. Pure MAS와 NetAlly의 차이는 도구 바인딩의 활성화 여부만이다. 따라서 $\Delta(\text{구조}) = \text{Pure MAS} - \text{Single}$ 은 에이전트 구조(질문 분해 + 다단계 추론)의 효과를, $\Delta(\text{도구}) = \text{NetAlly} - \text{Pure MAS}$ 는 동일 구조에서 도구 통합의 효과를 각각 분리한다. 다만, $\Delta(\text{구조})$ 에는 질문 분해와 컨텍스트 재구성이 혼재하므로 순수 ablation이 아닌 준실험(quasi-experiment) 설계임을 유의한다.

D. 오류 분석

L4/L5 답은 무작위 표본을 추출하여 오류 유형별로 분류한다(Table XII). 분석의 목적은 단순 정확도 비교를 넘어, 모델이 어떤 지점에서 추론을 중단하거나 잘못된 경로를 구성하는지 정성적으로 파악하는 데 있다. 특히 빈 응답 또는 추론 회피, 추가 홑 오류, IP-노드 매핑 실패, 형식 오류, 컨텍스트 포화는 L4/L5 성능 저하를 해석하는 핵심 후보 유형으로 간주한다. 최종 원고에서는 각 유형의 비율과 대표 사례를 함께 제시할 예정이다.

VI. DISCUSSION

A. 시뮬레이션 장벽의 원인

Section V-D의 오류 분석은 시뮬레이션 장벽이 정보의 부재가 아니라 계산의 불안정성에서 비롯됨을 시사한다. 설정 파일에는 OSPF cost, BGP neighbor, 인터페이스 IP와 같이 경로 계산에 필요한 정보가 이미 포함되어 있다. 그러나 이러한 정보를 정답으로 변환하려면 다수 노드에 걸친 SPF 계산과 VRF별 포워딩 상태 구성이 필요하며, 이는 단순한 텍스트 회상과는 다른 계산적 추론을 요구한다. 토큰 예측 방식의 LLM은 이 과정에서 중간 단계 오류를 누적시키며, 그 결과는 주로 빈 응답과 추론 회피, 또는 중간 홑을 잘못 삽입하는 추가 홑 오류의 형태로 나타난다. 이러한 실패 패턴이 파라미터 규모와 아키텍처에 관계없이 반복된다는 점은, 본 연구가 관찰한 한계가 개별 모델의 성능 차이보다 토큰 예측 패러다임 자체의 구조적 제약에 가깝다는 해석을 뒷받침한다. Mondal et al. [?]이 GPT-4에서도 라우터 설정 합성에 검증기 피드백 루프가 필요하다고 보고한 결과는 이러한 관찰과 맥을 같이한다. 따라서 네트워크 운영 관점에서

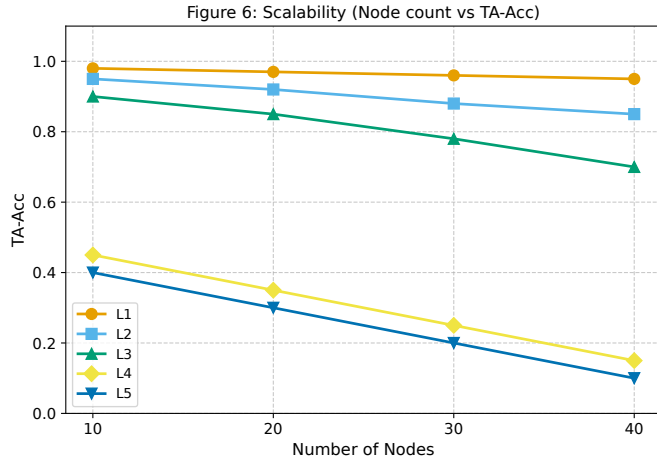


Fig. 5. 토폴로지 규모에 따른 레벨별 TA-Acc 변화. 이 그림은 노드 수 증가에 따라 각 레벨의 성능이 어떻게 달라지는지 비교하기 위한 것이다.

TABLE XI
3-WAY 비교 결과

레벨	Single	Pure MAS	NetAlly	Δ (구조)	Δ (도구)
L1	TBD	TBD	TBD		
L2	TBD	TBD	TBD		
L3	TBD	TBD	TBD		
L4	TBD	TBD	TBD		
L5	TBD	TBD	TBD		

TABLE XII
도구 호출 분석

레벨	주요 도구	호출 성공률	TA-Acc (성공)	TA-Acc (실패)
L4	Batfish traceroute	TBD	TBD	TBD
L5	Batfish fork_snapshot	TBD	TBD	TBD

TABLE XIII
L4/L5 오류 유형 분류

오류 유형	비율	근본 원인
빈 응답 / 추론 회피	TBD	경로 계산 자체를 시도하지 못하거나 “확인 불가”로 회피
추가 홉 오류	TBD	3홉 이상에서 중간 노드 누락 또는 불필요한 노드 추가
IP-노드 매핑 실패	TBD	목적지 IP의 소속 장비를 파악하지 못해 경로 과연장
형식 오류	TBD	path/set 포매팅 실패로 파싱 불가
Context 포화	TBD	대규모 config 입력 시 답변 퇴화

서 L1~L3 수준의 사실 추출과 정적 확인에는 LLM을 직접 활용할 수 있지만, L4/L5 수준의 동작 추론에는 정형 분석 도구의 병행이 사실상 불가피하다.

B. 구조 효과와 도구 효과의 분리

기존 연구 [?], [?]는 Multi-Agent 도입에 따른 성능 향상을 보고하지만, 그 향상의 원천이 에이전트 구조인지 연결된 도구인지까지는 분리해 설명하지 않는다. 이 구분은 시스템 설계 관점에서 중요하다. 만약 성능 향상의 핵심이 구조적 분업이 아니라 도구 접근성에 있다면, 복잡한 다중 에이전트 아키텍처를 구축하는 것보다 더 단순한 구조에 적절한 네트워크 분석 도구를 연결하는 편이 실용적으로 더 타당할 수 있기 때문이다.

본 연구의 3-way 비교는 바로 이 지점을 겨냥한다. Δ (구조)는 도구 없이 에이전트 구조만 추가했을 때의 효과를, Δ (도구)는 동일한 구조에 도구를 통합했을 때의 추가 효과를 나타낸다. L1~L3에서는 질문 분해와 답변 통합 과정이 일부 이점을 제공할 수 있으나, L4/L5에서는 하위 질문 자체가 traceroute나 what-if 분석과 같은 시뮬레이션 절차를 필요로 한다. 따라서 구조만으로는 한계를 넘기 어렵고, 실제 성능 개선은 도구 호출과 그 결과 해석이 결합될 때 비로소 나타난다. 이 구간에서 LLM의 역할은 직접적으로 경로를 계산하는 주체라기보다, 어떤 도구를 호출할지 선택하고 그 결과를 사용자 질의에 맞게 해석하는 조정자로 재정의된다.

C. 파이프라인의 범용성과 실용적 가치

이중 경로 파이프라인은 특정 토폴로지에 종속되지 않도록 설계되었다. 4개 Lab에 동일한 생성 절차를 적용한 결

과, 코드 수정 없이 메트릭 커버리지가 52%(Lab-A, 10노드)에서 67%(Lab-B), 88%(Lab-C), 97%(Lab-D, 40노드)까지 확장되었다. Lab-A에서 커버리지가 낮은 이유는 L2VPN, QoS, NetFlow 등 해당 토폴로지에 설정되지 않은 프로토콜 관련 메트릭이 비활성 상태이기 때문이며, 토폴로지 복잡도가 증가하면 활성 메트릭 수가 자동으로 늘어난다. QA/노드 비율도 84~126 범위에서 비교적 안정적으로 유지되었다. 이는 파이프라인이 단일 예제에 맞춘 일회성 구성이라기보다, 토폴로지 규모와 프로토콜 구성이 바뀌더라도 유사한 방식으로 적용될 수 있음을 보여준다. 실무적으로는 운영 조직이 자사 설정 파일만 제공하더라도 LLM 기반 도구의 사전 성능 평가, 변경 이후의 회귀 테스트, 또는 특정 운영 태스크에 대한 위험 점검에 이 프레임워크를 활용할 수 있다.

D. 타당성 위협

벤더 종속성. 실험은 Cisco IOS로 한정하였다. 이는 벤더 간 구분 차이와 운영 관례의 영향을 배제하여 비교 조건을 단순화하기 위한 선택이다. Batfish가 벤더 독립적 데이터 모델을 사용하므로 파이프라인과 난이도 체계 자체는 Juniper JUNOS나 Arista EOS로 확장 가능하지만, 본 논문의 결과는 우선 Cisco IOS 환경에서 해석하는 것이 적절하다.

Ground Truth 독립성. L4/L5의 Ground Truth와 NetAlly의 핵심 추론 도구가 모두 Batfish에 의존하므로, NetAlly의 L4/L5 정확도는 부분적으로 “올바른 Batfish API를 적절한 파라미터로 호출했는가”를 반영한다. 본 연구는 이러한 순환성을 통제하기 위해 세 가지 조치를 취하였다. 첫째, PNETLab 실장비 CLI와의 교차 검증으로 Batfish

출력의 신뢰성을 확인하였다(Section III-E, Method 3). 둘째, 순환성에서 자유로운 L1~L3에서도 유사한 성능 격차 패턴이 나타나는지 함께 확인하였다. 셋째, 도구 호출 성공/실패별 TA-Acc를 분리 보고하여(Table XI) “API 호출 성공”과 “추론 품질”을 구분하고자 하였다. 다만 Batfish 외의 독립 오라클, 예를 들어 실장비 기반 대규모 자동 검증을 통한 완전한 순환 해소는 향후 과제로 남아 있다.

토폴로지 규모. 본 연구의 최대 규모는 40노드로, 대규모 엔터프라이즈나 통신사업자 환경 전체를 대표하기에는 제한적이다. 그럼에도 QA/노드 비율이 비교적 안정적으로 유지되었고 Config Generator가 임의 규모의 토폴로지를 지원한다는 점은, 본 접근이 더 큰 환경으로 확장될 가능성을 보여준다. 다만 그 확장이 실제로 어떤 비용과 성능 저하를 수반하는지는 후속 실험으로 확인할 필요가 있다.

VII. CONCLUSION

본 연구는 “LLM은 네트워크 설정 파일을 얼마나 정확하게 해석하고 분석할 수 있는가?”라는 질문에 답하기 위해 벤치마크 NetConfigQA 2.0, 평가 지표 TA-Acc, 도구 통합 Multi-Agent 시스템 NetAlly를 제안하였다. 이중 경로 파이프라인은 설정 파일만으로 QA를 자동 생성하며, 4개 토폴로지(10~40 노드, 9,462 QA)에 코드 수정 없이 적용되었고 3중 GT 검증에서 98~99.9%의 일치율을 보였다. 6개 LLM 평가 결과, 사실 추출(L1~L3)에서는 모델 간 뚜렷한 성능 차이가 나타났으나 경로 추론과 장애 분석(L4/L5)에서는 모든 모델이 시뮬레이션 장벽에 직면하였다. 3-way 비교에서 MAS 구조만으로는 L4/L5가 개선되지 않았으나 네트워크 도구 통합 시 유의미한 변화가 관찰되어, 성능 향상의 주된 원인이 도구 통합에 있음을 확인하였다. TA-Acc는 BERTScore와 Exact Match가 구분하지 못하는 모델 간 차이를 식별하였다.

향후 연구로 L6 트러블슈팅 확장, 멀티 벤더(Juniper, Arista) 지원, 설정 생성 태스크 평가, 외부 벤치마크와의 교차 적용을 계획한다.

ACKNOWLEDGMENT

[TODO]

Hyeonjeong Lee [TODO]

Yujin Park [TODO]

Cheoneum Park Cheoneum Park is a Member of IEEE. [TODO]